

DOCUMENT DE SYNTHÈSE : CONCEPTION ET DÉVELOPPEMENT D'UN JEU SOKOBAN

Auteur : Antoine Meunier

Formation : BUT Informatique (Première année, Groupes 4B)

Cadre : Projet Final d'Interface Homme-Machine (IHM) et Développement d'Applications

1. Introduction

Ce document présente la synthèse technique du projet Sokoban, un jeu de réflexion classique implémenté en Java et doté d'une interface graphique moderne développée avec la bibliothèque JavaFX. L'objectif principal de ce projet consistait à allier la rigueur de la programmation orientée objet (POO) à une approche ergonomique de l'interface utilisateur. La conception logicielle a été pensée pour garantir la modularité du code, la fluidité des interactions et une gestion optimale des ressources de la machine.

2. Palier visé : Validation du Palier B

Le projet atteint et valide l'ensemble des exigences du **Palier B**. L'application intègre un moteur capable de gérer une suite de 10 niveaux aux configurations et difficultés variées.

L'élément clé de ce palier réside dans la **construction dynamique de la grille de jeu**.

Contrairement à une structure statique figée, le programme analyse en temps réel les dimensions (largeur et hauteur) définies dans les fichiers de configuration de chaque niveau. La scène et la matrice de composants graphiques s'ajustent ainsi de manière entièrement automatisée, garantissant l'extensibilité du jeu vers de nouveaux niveaux sans modification du code source.

3. Présentation de l'interface

L'ergonomie de l'application a été conçue pour offrir une expérience utilisateur intuitive et structurée :

- **Menu principal :** Point d'entrée clair permettant d'accéder aux différentes sections de l'application de manière fluide.
- **Sélection des modes :** Une interface dédiée offre le choix entre le mode *Aventure* (progression linéaire à travers les 10 niveaux) et le mode *Tutoriel* (apprentissage des mécaniques de base).
- **Écran de jeu :** Une zone d'action épurée et parfaitement lisible, où les informations textuelles (statistiques) et la grille de jeu sont agencées harmonieusement pour éviter toute surcharge cognitive.

4. Options implémentées

4.1. Options liées au jeu et au moteur Sokoban

- **Double compteur de performance** : En plus du suivi standard des déplacements du personnage, le moteur intègre un compteur spécifique pour le nombre de poussées de caisses. Cette distinction s'avère essentielle pour évaluer la qualité de la stratégie du joueur.
- **Titre dynamique de la fenêtre** : Le composant principal Stage met à jour son titre textuel de manière contextuelle lors de chaque changement de niveau, affichant explicitement le nom du niveau en cours.
- **Navigation inter-niveaux complète** : L'interface utilisateur intègre des commandes de transition (*Suivant / Précédent*) ainsi qu'un bouton de retour immédiat au menu de sélection des modes, fluidifiant le parcours de l'utilisateur.

4.2. Options liées à JavaFX et à l'IHM

- **Rendu par sprites 2D** : L'intégralité de l'affichage graphique repose sur l'utilisation d'objets `ImageView` exploitant des sprites de haute qualité, éliminant totalement l'usage de couleurs de remplissage CSS brutes.
- **Indicateur de progression en temps réel** : Un label dynamique affiche en permanence l'état d'avancement sous la forme "Caisses placées : X/Y". Ce composant est mis à jour instantanément à chaque modification de la matrice du jeu.
- **Optimisation de la mémoire via EnumMap** : Afin de garantir la fluidité des animations et d'éviter des accès disque répétitifs (I/O) à chaque mouvement, une structure de données `EnumMap` a été mise en place. Elle fait office de cache mémoire en préchargeant l'ensemble des textures nécessaires. Toutes les images sont ainsi instanciées une seule fois et forcées à une dimension stricte de **40x40 pixels**.

5. Éléments remarquables et choix d'architecture

5.1. Gestion de la superposition par calques (StackPane)

Pour assurer un rendu graphique propre et modulaire, le choix architectural s'est porté sur l'utilisation avancée du composant `StackPane`. Au lieu de dessiner une entité unique par case, chaque coordonnée de la grille superpose plusieurs couches graphiques distinctes :

1. **Arrière-plan** : La texture du sol.
2. **Plan intermédiaire** : La zone cible (si applicable).
3. **Premier plan** : Le joueur ou la caisse.

Cette décomposition s'appuie sur la gestion native du canal Alpha (transparence) des sprites, évitant les artefacts visuels et simplifiant grandement la gestion algorithmique des déplacements.

5.2. Orientation dynamique du sprite joueur

Afin de rendre l'expérience visuelle plus immersive, le personnage réagit de manière directionnelle aux entrées du clavier. En fonction de la touche pressée (Z, Q, S, D), le contrôleur intercepte l'événement et bascule instantanément l'affichage du joueur parmi 4 sprites distincts (haut, bas, gauche, droite).

5.3. Système de cycle de vie (Vies et Game Over)

Une mécanique de persistance a été développée via l'implémentation d'un système de 3 vies. Le moteur de jeu est capable de détecter une situation de blocage (*deadlock*, lorsqu'une caisse est poussée dans un coin non valide). En cas d'échec, le joueur perd une vie. Dès que le compteur atteint 0, l'affichage bascule vers un écran de *Game Over* persistant, interrompant la partie et redirigeant de force l'utilisateur vers le menu principal.

6. Conclusion et difficultés rencontrées

Le développement de ce projet Sokoban a permis de consolider des compétences avancées en programmation événementielle et en architecture d'IHM sous JavaFX. Deux défis techniques majeurs ont été surmontés :

1. **Gestion de scènes multiples sur un seul Stage** : L'enchaînement entre le menu principal, la sélection des modes et l'écran de jeu sans ouverture de fenêtres intempestives a nécessité la mise en place d'un contrôleur de navigation centralisé. Ce système permet d'intervertir le conteneur racine (*Scene*) tout en préservant le contexte et les données de l'application.
2. **Résolution des *NullPointerException* d'assets** : Le chargement initial des ressources graphiques générait des erreurs de pointage lors de l'exécution du fichier JAR. Ce problème a été résolu en appliquant de manière stricte les standards de la hiérarchie Maven, en déplaçant l'intégralité des sprites dans le dossier `src/main/resources` et en y accédant via la méthode robuste `getClass().getResource()`.